Lan_Yangrong_HW3

Name: yangrong lan
Github Username: CallistoAma
USC ID: 5561147067

1. Time Series Classification Part 1: Feature Creation/Extraction

(a) Download Data

Package imports and reproducibility settings.

```
from pathlib import Path

import numpy as np
import pandas as pd
from IPython.display import display

pd.set_option("display.max_columns", 12)
pd.set_option("display.width", 140)
RANDOM_STATE = 552
```

The supplied AReM archive is stored locally in `AReM/AReM`. The loader below handles the minor inconsistencies in the original files: one whitespace-delimited file, two files with a trailing comma, and one file with 479 rather than 480 rows. No signal values are discarded.

```
DATA_DIR = Path("AReM") / "AReM"
ACTIVITIES = [
    "bending1", "bending2", "cycling", "lying",
    "sitting", "standing", "walking"
]
SIGNALS = [
    "avg_rss12", "var_rss12", "avg_rss13",
    "var_rss13", "avg_rss23", "var_rss23"
]
COLUMNS = ["time", *SIGNALS]

def dataset_number(path):
    return int(path.stem.replace("dataset", ""))

def read_arem_file(path):
    # Read one AReM instance while tolerating the known formatting quirks.
    rows = []
    for raw_line in path.read_text(encoding="utf-8").splitlines():
        line = raw_line.strip()
        if not line or line.startswith("#"):
            continue
        parts = line.rstrip(",").split(",") if "," in line else line.split()
        if len(parts) != 7:
```

```
            raise ValueError(f"Expected 7 columns in {path}, got {len(parts)}")
        rows.append([float(value) for value in parts])
    return pd.DataFrame(rows, columns=COLUMNS)

instances = []
for activity in ACTIVITIES:
    paths = sorted((DATA_DIR / activity).glob("dataset*.csv"), key=dataset_number)
    for path in paths:
        number = dataset_number(path)
        test_numbers = {1, 2} if activity in {"bending1", "bending2"} else {1, 2, 3}
        instances.append({
            "activity": activity,
            "dataset": number,
            "split": "test" if number in test_numbers else "train",
            "path": path,
            "data": read_arem_file(path),
        })

print(f"Loaded {len(instances)} instances from {len(ACTIVITIES)} activities.")
print("Observation counts per instance:",
      pd.Series([len(item["data"]) for item in
instances]).value_counts().sort_index().to_dict())
```

```
Loaded 88 instances from 7 activities.
Observation counts per instance: {479: 1, 480: 87}
```

(b) Test and Train Data

```
manifest = pd.DataFrame([
    {
        "activity": item["activity"],
        "dataset": item["dataset"],
        "split": item["split"],
        "observations": len(item["data"]),
    }
    for item in instances
])

split_summary = pd.crosstab(manifest["activity"], manifest["split"])
display(split_summary)
print("Total train instances:", (manifest["split"] == "train").sum())
print("Total test instances:", (manifest["split"] == "test").sum())
```

split	test	train
activity		
bending1	2	5
bending2	2	4
cycling	3	12
lying	3	12
sitting	3	12
standing	3	12
walking	3	12

```
Total train instances: 69
Total test instances: 19
```

(c) Feature Extraction

i. Research

Common time-domain features for time-series classification include minimum, maximum, range, mean, median, variance, standard deviation, quantiles, interquartile range, root-mean-square value, energy, skewness, kurtosis, zero-crossing rate, slope/trend, and autocorrelation-related summaries.

This assignment specifies seven of them: minimum, maximum, mean, median, standard deviation, first quartile, and third quartile.

ii. Extraction

```
STATISTICS = {
    "min": lambda x: np.min(x),
    "max": lambda x: np.max(x),
    "mean": lambda x: np.mean(x),
    "median": lambda x: np.median(x),
    "std": lambda x: np.std(x, ddof=1),
    "1st_quart": lambda x: np.quantile(x, 0.25),
    "3rd_quart": lambda x: np.quantile(x, 0.75),
}

feature_rows = []
metadata_rows = []
for instance_id, item in enumerate(instances, start=1):
    row = {}
    for series_number, signal in enumerate(SIGNALS, start=1):
        values = item["data"][signal].to_numpy()
        for statistic_name, statistic in STATISTICS.items():
            row[f"{statistic_name}{series_number}"] = statistic(values)
    feature_rows.append(row)
    metadata_rows.append({
        "instance": instance_id,
        "activity": item["activity"],
        "dataset": item["dataset"],
        "split": item["split"],
    })

feature_table = pd.DataFrame(feature_rows, index=range(1, len(feature_rows) + 1))
feature_table.index.name = "instance"
metadata = pd.DataFrame(metadata_rows).set_index("instance")

print("Feature matrix shape:", feature_table.shape)
all_features = pd.concat([metadata, feature_table], axis=1)
display(all_features.iloc[:5, :10].round(3))
print("Preview: first 5 rows and first 10 of 45 metadata/feature columns.")
```

```
Feature matrix shape: (88, 42)
```

instance	activity	dataset	split	min1	max1	mean1	median1	std1	1st_quart1	3rd_quart1
1	bending1	1	test	37.25	45.00	40.625	40.50	1.477	39.25	42.00
2	bending1	2	test	38.00	45.67	42.813	42.50	1.436	42.00	43.67
3	bending1	3	train	35.00	47.40	43.954	44.33	1.559	43.00	45.00
4	bending1	4	train	33.00	47.75	42.180	43.50	3.671	39.15	45.00
5	bending1	5	train	33.00	45.75	41.678	41.75	2.243	41.33	42.75

Preview: first 5 rows and first 10 of 45 metadata/feature columns.

iii. Standard Deviation

```
# Point estimates use the sample standard deviation (ddof=1). The 90%
# confidence intervals are percentile bootstrap intervals based on 10,000
# resamples of the 88 instances.
values = feature_table.to_numpy()
n_instances, n_features = values.shape
n_bootstrap = 10_000
rng = np.random.default_rng(RANDOM_STATE)
bootstrap_std = np.empty((n_bootstrap, n_features))

block_size = 250
for start in range(0, n_bootstrap, block_size):
    stop = min(start + block_size, n_bootstrap)
    indices = rng.integers(0, n_instances, size=(stop - start, n_instances))
    bootstrap_std[start:stop] = values[indices].std(axis=1, ddof=1)

ci_lower, ci_upper = np.percentile(bootstrap_std, [5, 95], axis=0)
std_summary = pd.DataFrame({
    "standard_deviation": feature_table.std(ddof=1),
    "90% CI lower": ci_lower,
    "90% CI upper": ci_upper,
})

display(std_summary.round(4))
```

	standard_deviation	90% CI lower	90% CI upper
min1	9.5700	8.2622	10.7614
max1	4.3944	3.3280	5.2849
mean1	5.3357	4.7040	5.8864
median1	5.4401	4.7903	6.0043
std1	1.7722	1.5722	1.9500
1st_quart1	6.1536	5.5816	6.6524
3rd_quart1	5.1389	4.3319	5.8409
min2	0.0000	0.0000	0.0000
max2	5.0627	4.6228	5.4022
mean2	1.5742	1.3941	1.7068
median2	1.4122	1.2361	1.5440
std2	0.8841	0.8026	0.9418
1st_quart2	0.9464	0.8294	1.0359
3rd_quart2	2.1253	1.8914	2.2967
min3	2.9565	2.7601	3.1079
max3	4.8751	4.1702	5.4573
mean3	4.0084	3.4163	4.4918
median3	4.0364	3.4388	4.5351
std3	0.9467	0.7614	1.1198
1st_quart3	4.2207	3.6305	4.7049
3rd_quart3	4.1716	3.5434	4.6940
min4	0.0000	0.0000	0.0000
max4	2.1836	1.9741	2.3595
mean4	1.1661	1.0740	1.2225
median4	1.1456	1.0522	1.2017
std4	0.4582	0.4207	0.4866
1st_quart4	0.8436	0.7726	0.8911
3rd_quart4	1.5525	1.4300	1.6284
min5	6.1240	4.4033	7.5110
max5	5.7412	4.7406	6.5541
mean5	5.6756	4.4016	6.7385
median5	5.8138	4.4902	6.9103
std5	1.0249	0.8132	1.2158
1st_quart5	6.0965	4.7701	7.2176
3rd_quart5	5.5317	4.3398	6.5283
min6	0.0458	0.0000	0.0785
max6	2.5189	2.2508	2.7639
mean6	1.1548	1.0601	1.2163
median6	1.0865	0.9925	1.1487
std6	0.5176	0.4781	0.5455
1st_quart6	0.7586	0.6882	0.8087
3rd_quart6	1.5236	1.3985	1.6015

iv. Select Features

I select **mean, standard deviation, and maximum**. Mean describes the overall signal level, standard deviation captures within-instance variability, and the maximum records upper-end intensity. Together they represent location, dispersion, and an extreme while avoiding the strong redundancy of choosing both mean and median.

2. ISLR 3.7.4

(a) Comparing males and females

The correct answer is **(iii): For a fixed value of IQ and GPA, males earn more on average than females provided that the GPA is high enough.**

Holding IQ and GPA fixed, the fitted difference is:

```
fitted salary (female) - fitted salary (male) = 35 - 10 * GPA
```

Therefore females have the higher fitted salary when GPA < 3.5, the fitted salaries are equal at GPA = 3.5, and males have the higher fitted salary when GPA > 3.5.

(b) Prediction

For a female with IQ = 110 and GPA = 4.0:

```
50 + 20(4.0) + 0.07(110) + 35(1) + 0.01(4.0)(110) - 10(4.0)(1) = 137.1
```

The predicted starting salary is **$137,100**.

(c) Interpreting the interaction coefficient

False. A small coefficient does not by itself imply weak evidence for an interaction. Statistical evidence depends on the coefficient relative to its standard error (or its p-value), neither of which is given. The scale of the predictors also matters: here the interaction contribution for GPA = 4 and IQ = 110 is 0.01(4)(110) = 4.4 thousand dollars, which is not automatically negligible.

(d) Conclusion

The model's interaction terms make the effect of GPA depend on gender and the effect of GPA depend on IQ. Main-effect coefficients therefore should not be interpreted in isolation from the interactions.

3. ISLR 3.7.3 - Extra Practice

Not submitted, as specified in the assignment.

4. ISLR 3.7.5 - Extra Practice

Not submitted, as specified in the assignment.